

Python Advance Assignment 1

Submitted By :-

Naman Patel

[**Note** :- This PDF contains theory questions, while their programs are also been uploaded on github]

Iterator and Generator :-

14) Explain the difference between iterator and generator with a small example.

Iterator :-

- iterator is an object that returns values one by one using **next()** function.
- It is created using a **class** and must define **__iter__()** and **__next__()** methods
- We can provide custom behaviour and controls how values are generated
- More code is required and Stops by raising StopIteration

Examples :- printing number using iterator

class NumberIterator:

```
def __init__(self, limit):  
    self.num = 1  
    self.limit = limit
```

```
def __iter__(self):  
    return self
```

```
def __next__(self):  
    if self.num > self.limit:  
        raise StopIteration
```

```
    value = self.num  
    self.num += 1  
    return value
```

```
iterator = NumberIterator(3)  
for num in iterator:  
    print(num)
```

Generator :-

- generator is a function that returns values one by one using the **yield** keyword
- Created using a **function** and Uses **yield** instead of return.
- It is used to generate sequential values
- Less code is required

Examples :- printing number using generator

```
def number_generator(limit):  
  
    for number in range(1, limit + 1):  
        yield number  
  
generator = number_generator(3)  
for num in generator:  
    print(num)
```

Testing and Debugging :-

35) Explain the advantages of using an IDE debugger over print statements.

Advantages of Using an IDE Debugger Over Print Statements :-

- **Step by step execution** :- A debugger allows us to execute code one line at a time and observe how it works
- **Inspect variables** :- We can view variable values at any point without adding multiple print statements
- **Easier bug detection** :- It helps to identify the exact location and cause of errors.
- **Breakpoints** :- We can pause execution at any line and check what is happening
- **No code modification** :- Unlike print statements, we don't need to add and remove debugging code
- **Better for large projects** :- Debuggers are more efficient when working with complex applications

Example :-

Using print statements:

```
total = a+b
print(total)

result = total/100
print(result)
```

Using a debugger:

- Set a breakpoint.
- Run the program.
- Inspect variables directly in the IDE

Packaging :-

38) Explain the difference between module and package with an example.

Module :-

- module is a single Python file saved with extension **.py**
- It contains Python code such as functions, classes or variables.
- It helps organize and reuse code

Examples :-

(calculation.py)

```
def add(a, b):
    return a + b

def subtract(a,b):
    return a - b
```

(custom.py)

```
import calculation

print(calculation.add(10,20))

print(calculation.subtract(10,20))
```

Package :-

- package is a folder containing multiple modules
- it contains related modules or python files
- It helps to organize the structure of project

Examples :-

calculator package -> addition.py , subtraction.py

operation package -> perform_calculation.py

(addition.py)

```
def add(a, b):  
    return a + b
```

(subtraction.py)

```
def subtract(a, b):  
    return a - b
```

(perform_calculation.py)

```
from calculator.addition import add  
print(add(10, 20))
```